SecureChat: comunicación segura con criptografía híbrida

Taller de Criptografía Aplicada — Informe y documentación

Randall Bolaños (carné _____) Javier Tenorio (carné _____)

CE-1105 Seguridad de la Información ___ de octubre de 2026

Índice

1. Resumen de la solución **1**

2. Protocolo implementado **1**
2.1. Algoritmos y tamaños de clave 1
2.2. Handshake 2
2.3. Estructura de los mensajes transmitidos 2

3. Desarrollo y evidencias **2**
3.1. Pasos 1–3: conexión, material asimétrico y handshake 2
3.2. Pasos 4–5: mensajes protegidos y conversación bidireccional 3
3.3. Paso 6: detección de manipulación 5
3.4. Pruebas automáticas 6

4. Verificación de requisitos mínimos **7**

5. Preguntas de cierre **7**

6. Conclusiones y limitaciones **8**

Video de demostración: https://ENLACE-DEL-VIDEO
Código fuente: `securechat.py`, `mitm_proxy.py`, `test_securechat.py` (ver `README.md` para instalar y ejecutar).

1. Resumen de la solución

SecureChat es una aplicación de consola en Python 3 que se ejecuta dos veces en la misma computadora: una instancia *escucha* (Bob) y otra se *conecta* (Alice) mediante un socket TCP real sobre `127.0.0.1`. Antes del primer mensaje ambas completan un *handshake* asimétrico (RSA-2048) que establece una clave de sesión AES-256 aleatoria sin transmitirla nunca en claro; a partir de ahí cada mensaje se protege con AES-256-GCM, que cifra y autentica. Se usa la biblioteca `cryptography` (versión 43).

Además se incluye `mitm_proxy.py`, un proxy que se intercala entre ambos procesos, imprime los bytes que realmente cruzan la red y puede alterar un byte del ciphertext para demostrar la detección de manipulación, y un conjunto de 9 pruebas automáticas.

2. Protocolo implementado

2.1. Algoritmos y tamaños de clave

Elemento	Elección
Par asimétrico (cada participante)	RSA, $e = 65537$, **2048 bits**. Privada en `claves/<nombre>_privada.pem` con permisos 600.
Protección de la clave de sesión	**RSA-OAEP** con MGF1-SHA256 y SHA-256 (256 B de salida).
Autenticación del iniciador	Firma **RSA-PSS-SHA256** de Alice sobre (resumen del handshake ‖ clave cifrada).
Clave de sesión	32 bytes de `os.urandom` (CSPRNG), nueva en cada ejecución: **AES-256**.
Cifrado de mensajes	**AES-256-GCM**: nonce de 96 bits, tag de 128 bits.
Nonce	prefijo de dirección (4 B) ‖ contador de 64 bits; nunca se repite.
Datos asociados (AAD)	`"SecureChat-v1"` ‖ tipo de trama ‖ H, con H = SHA256(`"SecureChat-v1"`‖HELLO$_A$‖HELLO$_B$).

2.2. Handshake

```
Alice (conectar)                                    Bob (escuchar)
HELLO {nombre, pubA}   ---------------------------------->   (publico)
                       <----------------------------------  HELLO {nombre, pubB}
K = os.urandom(32)                  <- clave AES-256 de sesion
C = RSA-OAEP(pubB, K)               <- solo privB la puede abrir
S = RSA-PSS-Sign(privA, H || C)     <- prueba de que la envia Alice
KEY_EXCHANGE  C || S   ---------------------------------->  Verify(pubA, S)
                                                          K = RSA-OAEP^-1(privB, C)
                       <----------------------------------  SESSION_OK = AES-GCM_K("SESSION-OK")
verifica SESSION_OK
====== ambos poseen K  ->  "SESION SEGURA ESTABLECIDA"  ->  chat AES-256-GCM ======
```

La clave de sesión no está escrita en el código y solo cruza el socket cifrada con la clave pública de Bob. Las claves privadas jamás se transmiten ni se imprimen. El mensaje `SESSION_OK` sirve de confirmación: al estar cifrado y autenticado con K, Alice comprueba que Bob obtuvo exactamente la misma clave. De forma opcional, `--clave-par <pem>` fija la clave pública esperada del otro extremo (*pinning*) para impedir que un intermediario sustituya las claves públicas en el HELLO.

2.3. Estructura de los mensajes transmitidos

Toda trama tiene la forma `[longitud 4 B][tipo 1 B][payload]`:

Tipo	Nombre	Payload	¿Protegido?
`0x01`	HELLO	JSON {nombre, clave pública PEM}	No (es público)
`0x02`	KEY_EXCHANGE	RSA-OAEP(K) 256 B ‖ firma PSS 256 B	RSA
`0x03`	SESSION_OK	nonce ‖ ciphertext ‖ tag	AES-256-GCM
`0x10`	DATA	nonce 12 B ‖ ciphertext n B ‖ tag 16 B	AES-256-GCM
`0x11`	BYE	nonce ‖ ciphertext ‖ tag	AES-256-GCM

Ejemplo real: la trama del mensaje `"Hola Bob"` (41 bytes) tomada de la ejecución:

```
0000 0025 | 10 | 0000 0001 0000 0000 0000 0000 | 89e8 19b0 2b49 bb5a | f42e c623 7b1f 7be7 c272 013e 490a 77a8
longitud   tipo   nonce (direccion=1, ctr=0)     ciphertext (8 B)      tag GCM (16 B)
```

Los bytes ASCII de "Hola Bob" (`486f 6c61 2042 6f62`) no aparecen en la trama.

Nonce. Los 4 primeros bytes identifican la dirección (`00000001` Alice→Bob, `00000002` Bob→Alice) y los 8 restantes son un contador que solo crece. Así ningún extremo repite un nonce propio y los nonces de ambos extremos nunca colisionan aunque compartan K. El receptor exige además contador estrictamente creciente (anti-*replay*).

3. Desarrollo y evidencias

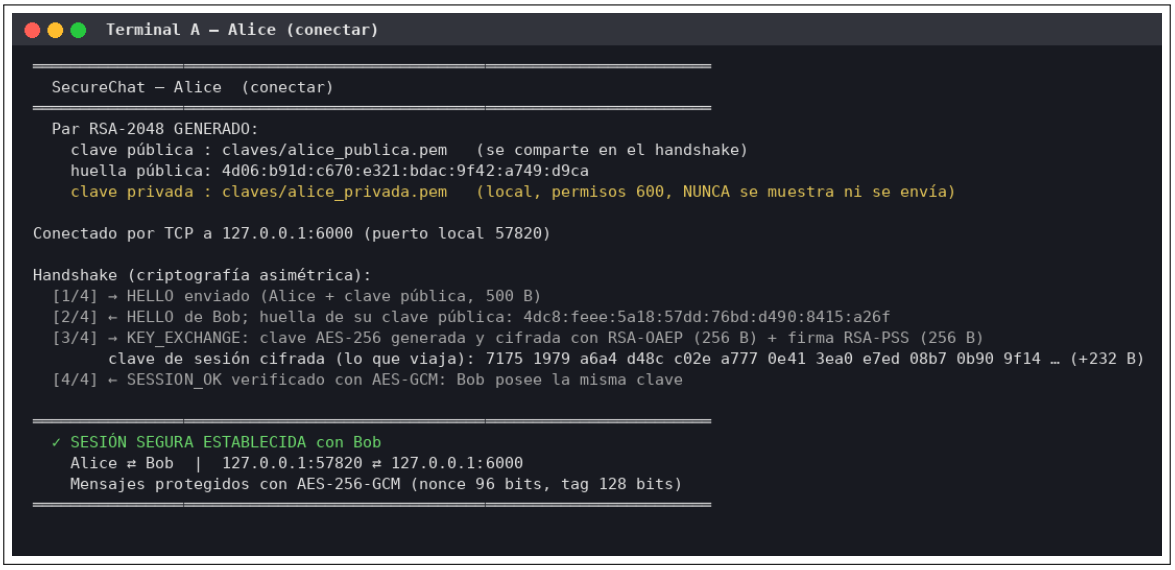
La ejecución de las capturas usó tres terminales: Bob (`escuchar`, puerto 5000), el proxy (puerto 6000 → 5000, alterando la 7.ª trama DATA) y Alice (`conectar` al 6000). Las salidas completas están en la carpeta `evidencia/`.

3.1. Pasos 1–3: conexión, material asimétrico y handshake

Cada instancia genera (o carga) su par RSA-2048, muestra solo la ruta y la huella de la clave pública (nunca el contenido de la privada), establece la conexión TCP indicando con quién está conectada y completa los 4 pasos del handshake antes de habilitar el chat.



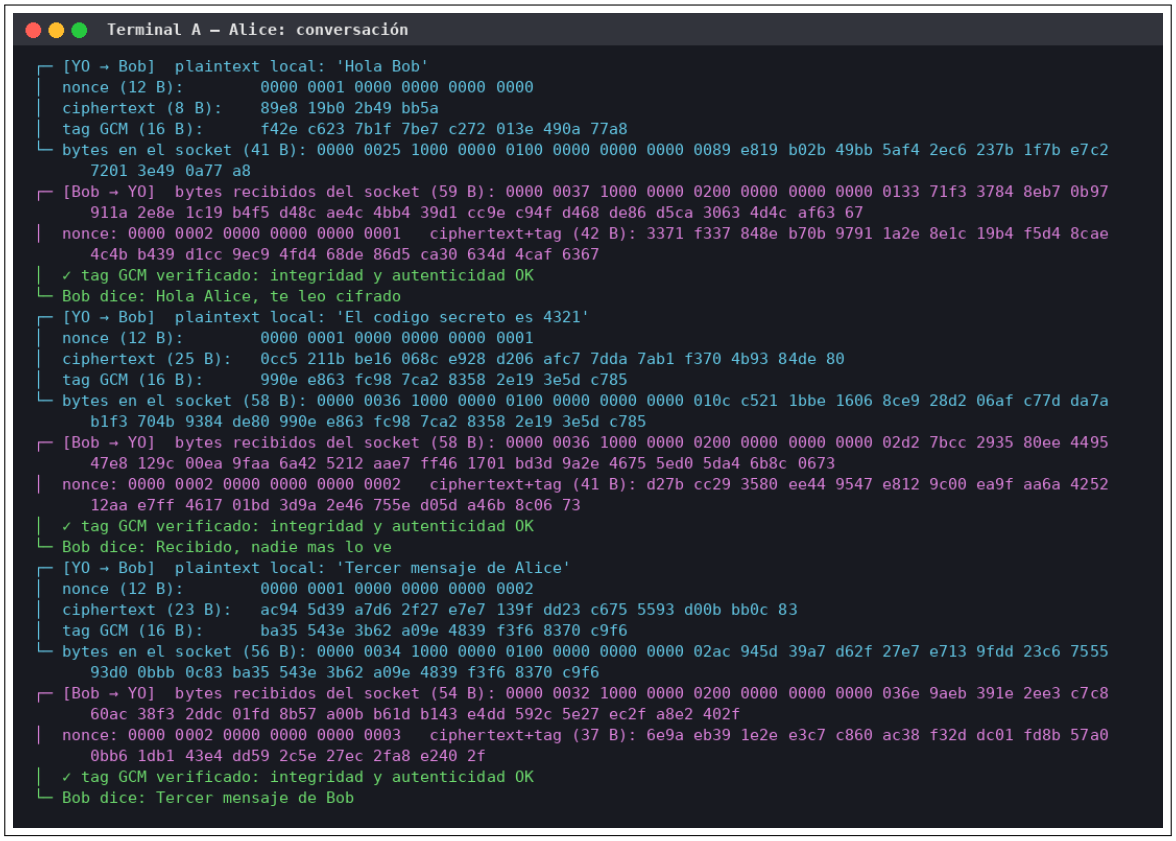
Bob: claves, conexión TCP aceptada, handshake y "sesión segura establecida".



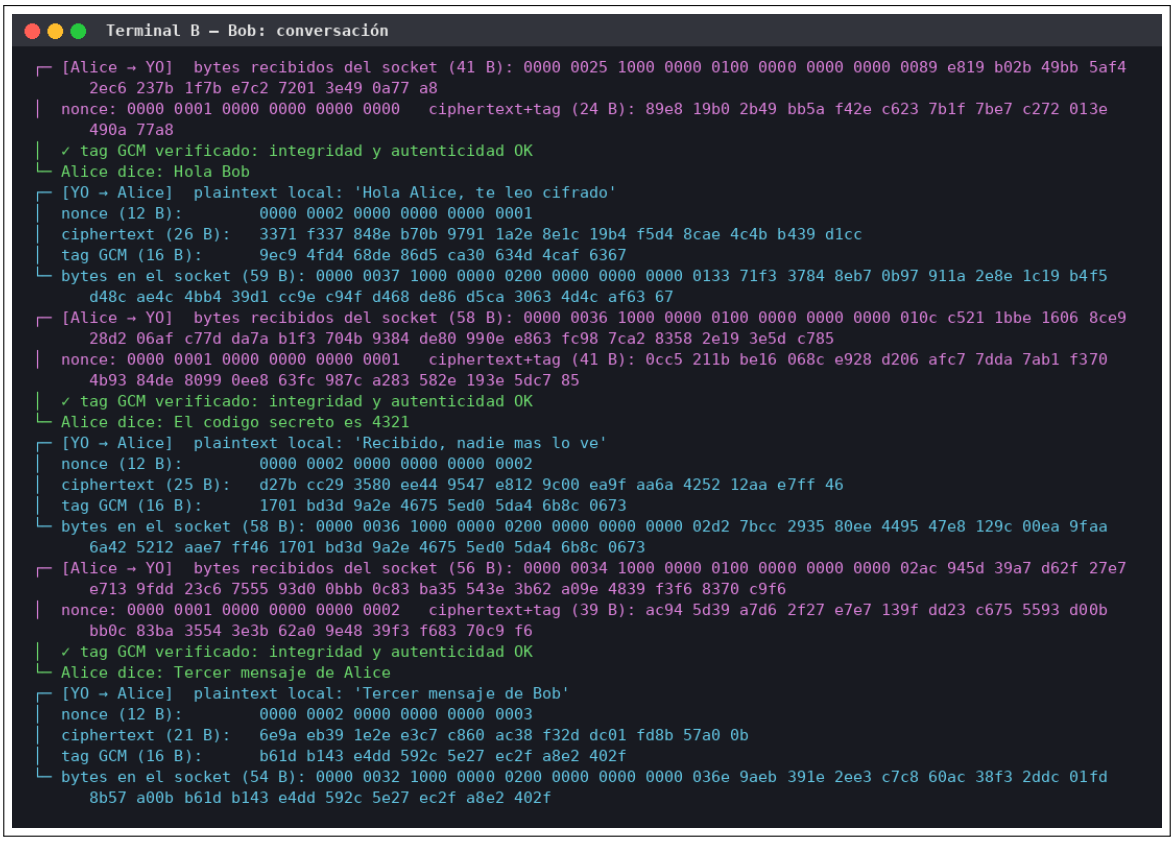
Alice: la clave de sesión se genera en tiempo de ejecución y solo viaja cifrada con RSA-OAEP (256 B).

3.2. Pasos 4–5: mensajes protegidos y conversación bidireccional

El emisor muestra el plaintext local, el nonce, el ciphertext, el tag y los bytes exactos escritos en el socket; el receptor muestra los bytes recibidos y, *solo tras verificar el tag*, el plaintext recuperado. Se enviaron tres mensajes en cada dirección; el contador del nonce avanza (0, 1, 2, . . .) en cada mensaje.

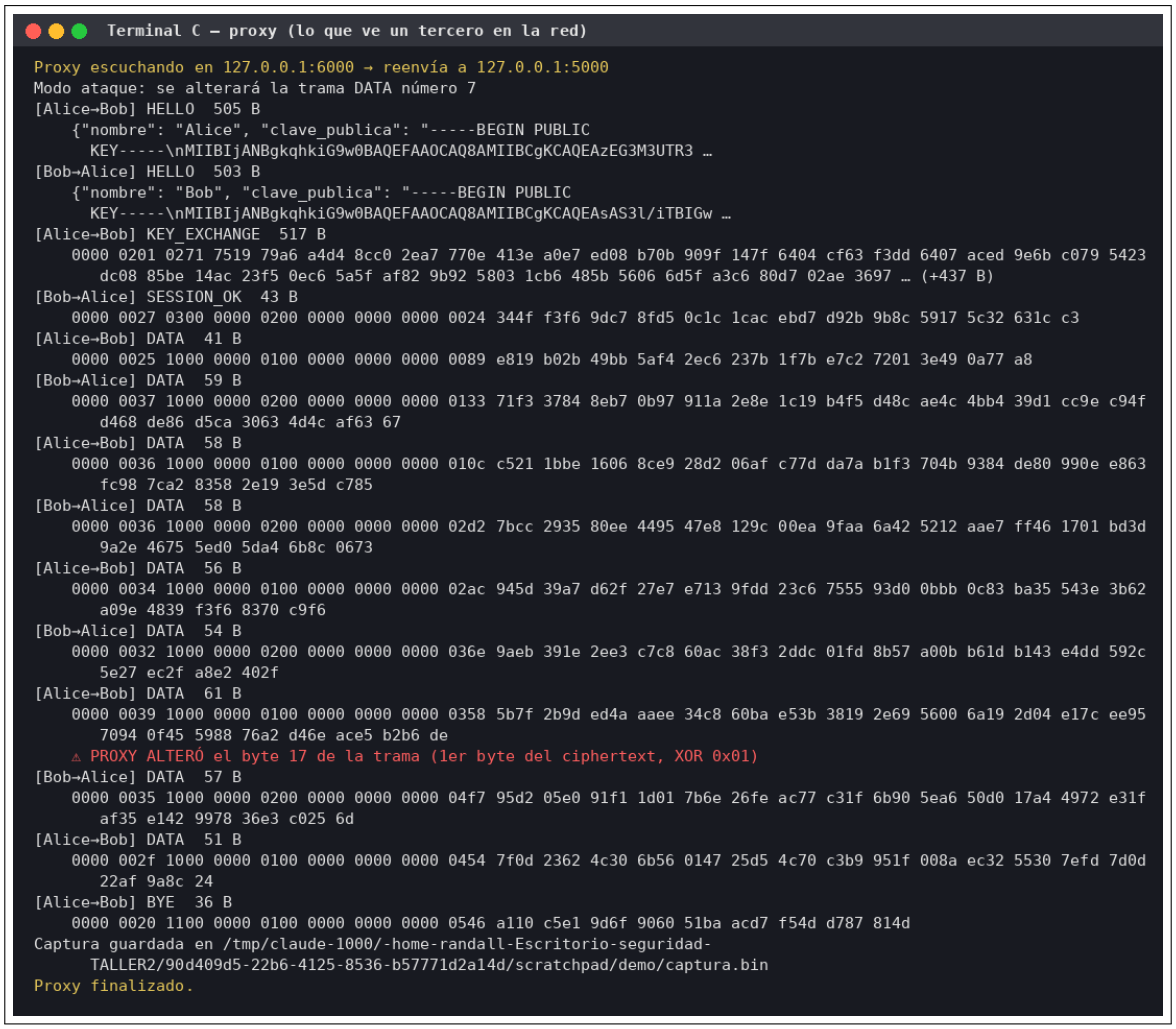


Terminal de Alice: mensajes propios (plaintext → ciphertext) y mensajes de Bob (ciphertext → plaintext verificado).



Terminal de Bob: los bytes recibidos son idénticos a los enviados por Alice.

Lo que ve un tercero en la red. El proxy imprime cada trama tal como cruza el socket. Solo los HELLO son legibles (contienen claves *públicas*); todo lo demás es ciphertext.



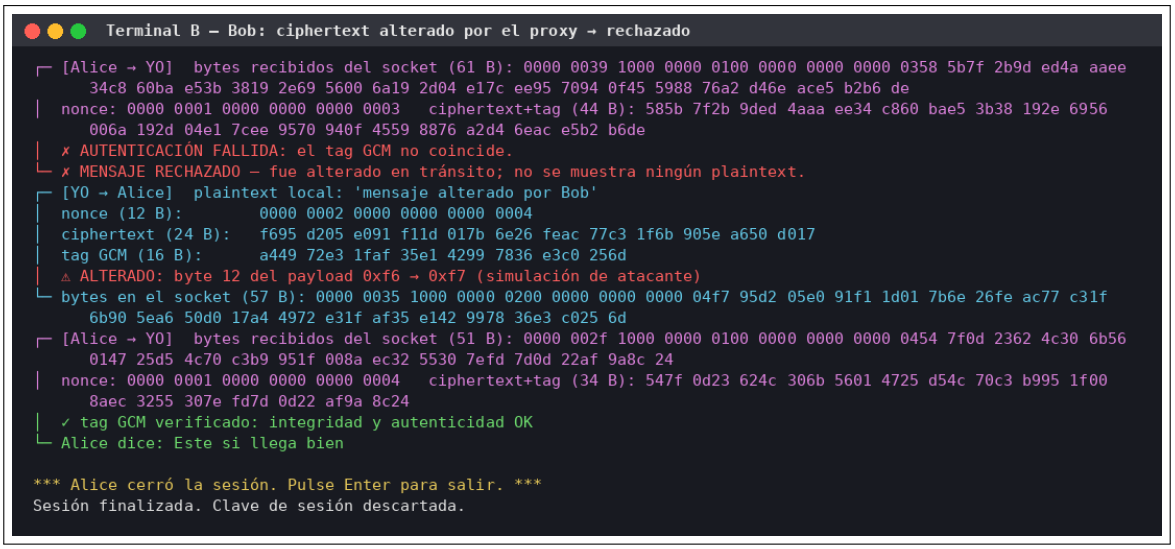
Tráfico capturado por el proxy: ningún mensaje del chat aparece en claro.

Sobre la captura binaria completa (`evidencia/captura.bin`):

```
$ grep -a -c "Hola Bob" captura.bin -> 0
$ grep -a -c "4321" captura.bin -> 0
$ grep -a -c "PRIVATE" captura.bin -> 0
```

3.3. Paso 6: detección de manipulación

Se hicieron dos pruebas: (a) el proxy invirtió un bit del primer byte del ciphertext de la trama "Transferir 100 colones a Bob" y (b) Bob usó `/alterar`, que modifica un byte del ciphertext antes de enviarlo. En ambos casos el receptor detecta que el tag GCM no coincide, **rechaza** el mensaje y no muestra ningún plaintext. El siguiente mensaje legítimo se acepta con normalidad.

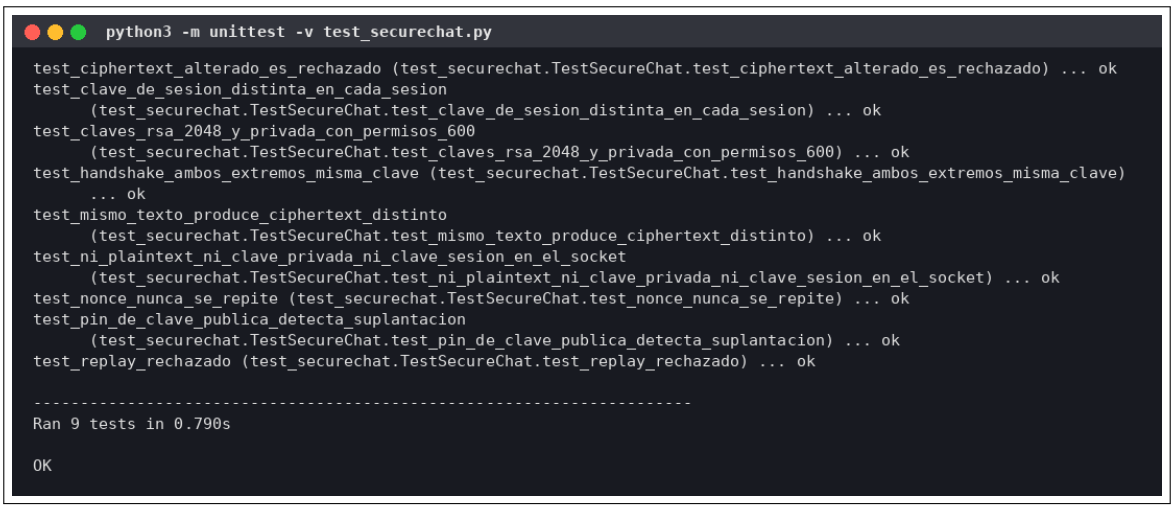


Bob rechaza la trama alterada por el proxy; después acepta "Este si llega bien".



Alice rechaza el mensaje que Bob alteró con `/alterar`.

3.4. Pruebas automáticas



Las 9 pruebas de requisitos de seguridad pasan.

4. Verificación de requisitos mínimos

	Requisito	Cómo se cumple
✓	Python 3 y sockets TCP/IP	`socket.AF_INET/SOCK_STREAM` sobre 127.0.0.1
✓	Biblioteca reconocida	`cryptography` (hazmat RSA, AESGCM)
✓	Handshake asimétrico	RSA-2048 OAEP + firma PSS
✓	Clave de sesión dinámica	`os.urandom(32)` por sesión; prueba *clave_distinta_en_cada_sesion*
✓	Ninguna privada por el socket	solo se envían públicas; prueba de captura y `grep PRIVATE` = 0
✓	Cifrado autenticado	AES-256-GCM en DATA, SESSION_OK y BYE
✓	Nonce correcto y no repetido	prefijo de dirección + contador; prueba con 2000 cifrados
✓	Rechazo de tag inválido	`InvalidTag` ⇒ "MENSAJE RECHAZADO"; prueba alterando cada byte
✓	Evidencia visible	plaintext local, ciphertext enviado, ciphertext recibido, plaintext recuperado
✓	No Base64/hex como cifrado	hex solo para mostrar bytes en pantalla
✓	No ECB	GCM (modo contador autenticado); mismo texto ⇒ ciphertexts distintos

5. Preguntas de cierre

1. **¿Qué problema resuelve la criptografía asimétrica durante el handshake?** El de la distribución de claves: Alice y Bob no comparten ningún secreto previo y el canal es observable. Con RSA-OAEP Alice cifra la clave de sesión con la clave *pública* de Bob, que puede conocer cualquiera, y solo la *privada* de Bob, que nunca sale de su equipo, puede recuperarla. Así acuerdan un secreto simétrico sin enviarlo nunca en claro. La firma RSA-PSS de Alice autentica además quién generó esa clave.

2. **¿Por qué la conversación utiliza criptografía simétrica después del handshake?** Porque AES-GCM es órdenes de magnitud más rápido que RSA, no tiene límite práctico de tamaño (RSA-OAEP-2048 solo cifra unos 190 B por operación y produce 256 B) y aporta autenticación integrada. RSA se usa una sola vez para acordar K, y la primitiva eficiente protege todos los mensajes: eso es la criptografía híbrida.

3. **¿Qué información puede observar un tercero que capture los paquetes?** IPs y puertos; el momento y el tamaño de cada trama, lo que deja inferir la longitud de cada mensaje; los HELLO, con los nombres y las claves *públicas*; la clave de sesión cifrada con RSA y la firma; y, por cada mensaje, el tipo, el nonce, el ciphertext y el tag. No puede ver el contenido, la clave de sesión ni las claves privadas (ver captura del proxy).

4. **¿Por qué el nonce puede viajar junto al ciphertext y por qué no debe repetirse?** El nonce no es secreto: solo garantiza que cada cifrado sea único, y la seguridad descansa en la clave. El receptor lo necesita para descifrar. Pero GCM cifra en modo contador: repetir un nonce con la misma clave reutiliza el mismo flujo de cifrado ($C_1 \oplus C_2 = P_1 \oplus P_2$, lo que filtra el contenido) y permite recuperar la subclave de autenticación GHASH y *falsificar* tags. Por eso usamos un contador monotónico con prefijo por dirección.

5. **¿Qué función cumple el authentication tag?** Es un código de autenticación de 128 bits que se calcula con K sobre el ciphertext y los datos asociados. El receptor lo verifica

antes de entregar el plaintext: si cambió un solo bit del nonce, el ciphertext, el tag o el contexto, la verificación falla y el mensaje se rechaza. Garantiza integridad y autenticidad (lo envió alguien que posee K). Cifrar sin autenticar permitiría modificar mensajes sin que nadie lo detectara.

6. **¿Qué ocurriría si la clave de sesión se transmitiera sin protección?** Cualquiera que capturara el tráfico obtendría K y podría descifrar toda la conversación y además fabricar o modificar mensajes con tags válidos, que el receptor aceptaría como auténticos. AES-GCM no aportaría ninguna seguridad. Codificarla en Base64 o hex no cambia nada: codificar no es cifrar.

7. **¿Qué ocurriría si un atacante obtiene una de las claves privadas?** Con la privada de *Bob* podría descifrar el KEY_EXCHANGE de cualquier sesión capturada, pasada o futura, obtener K y leerla completa, porque el transporte de clave con RSA no ofrece *forward secrecy*. También podría suplantar a Bob. Con la privada de *Alice* podría firmar intercambios falsos y suplantarla ante Bob, aunque no descifrar sesiones pasadas. Las mitigaciones son proteger las privadas (permisos, contraseña, HSM), revocarlas y regenerarlas, y usar un intercambio efímero (ECDHE/X25519) firmado, como TLS 1.3, para lograr *forward secrecy*.

6. Conclusiones y limitaciones

La práctica separa con claridad los dos papeles. La criptografía asimétrica resuelve una sola vez el problema de acordar un secreto sin canal seguro previo, y la simétrica autenticada protege eficientemente cada mensaje. La prueba de manipulación muestra que cifrar no basta: sin el tag, un atacante podría alterar el ciphertext sin ser detectado.

Limitaciones conscientes del diseño:

- La autenticidad de las claves públicas depende de comparar huellas o de usar `--clave-par`; no hay PKI.
- No hay *forward secrecy*.
- La longitud de los mensajes es observable.
- Las privadas se guardan sin contraseña; para la demo solo se protegen con permisos 600.